

MI APC API

Version 1.6

© 2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
1.0	● Initial release	09/09/2020
1.1	● Modified API	10/15/2020
1.2	● Add parameter in AGC config	12/17/2020
1.3	● Add parameter in ANR config	08/03/2021
1.4	● Add frequency domain DRC in AGC module	08/04/2021
1.5	● Add IaaApc_GetVadOut function	08/10/2021
1.6	● Add IaaApc_ApplyDigitalGain function	08/31/2021

目录

REVISION HISTORY	i
目录ii	
1. 概述	1
1.1. 算法说明	1
1.2. 关键词说明	1
1.3. 注意	1
2. API 参考	2
2.1. 功能模块 API	2
2.2. IaaApc_GetBufferSize	3
2.3. IaaApc_Init	4
2.4. IaaApc_Config	5
2.5. IaaApc_SetLowFreqCompressionRatioCurve	6
2.6. IaaApc_SetMidFreqCompressionRatioCurve	7
2.7. IaaApc_SetHighFreqCompressionRatioCurve	8
2.8. IaaApc_SetAgcFreqBand	8
2.9. IaaApc_GetNrResult	9
2.10. IaaApc_GetNrEqResult	10
2.11. IaaApc_GetVadOut	11
2.12. IaaApc_Run	12
2.13. IaaApc_Free	17
2.14. IaaApc_Reset	18
2.15. IaaApc_GetConfig	19
2.16. IaaApc_ApplyDigitalGain	20
2.17. IaaAnr_GetBufferSize	21
2.18. IaaAnr_Init	21
2.19. IaaAnr_Config	22
2.20. IaaAnr_Run	23
2.21. IaaAnr_Free	26
2.22. IaaAnr_Reset	27
2.23. IaaAnr_GetConfig	28
2.24. IaaEq_GetBufferSize	29
2.25. IaaEq_Init	29
2.26. IaaEq_Config	30
2.27. IaaEq_Run	31
2.28. IaaEq_Free	34
2.29. IaaEq_Reset	35
2.30. IaaEq_GetConfig	36
2.31. IaaAgc_GetBufferSize	37
2.32. IaaAgc_Init	37
2.33. IaaAgc_Config	38
2.34. IaaAgc_SetLowFreqCompressionRatioCurve	39
2.35. IaaAgc_SetMidFreqCompressionRatioCurve	40

2.36. IaaAgc_SetHighFreqCompressionRatioCurve.....	41
2.37. IaaAgc_SetAgcFreqBand.....	42
2.38. IaaAgc_Run.....	43
2.39. IaaAgc_Free.....	46
2.40. IaaAgc_Reset.....	47
2.41. IaaAgc_GetConfig.....	48
3. APC 数据类型.....	49
3.1. APC 模块相关数据类型定义.....	49
3.2. AudioApcBufferConfig.....	49
3.3. AudioProcessInit.....	50
3.4. APC_HANDLE.....	51
3.5. AudioAnrConfig.....	52
3.6. AudioEqConfig.....	53
3.7. AudioHpfConfig.....	54
3.8. AudioAgcConfig.....	55
3.9. AgcGainInfo.....	58
3.10. IAA_APC_SAMPLE_RATE.....	59
3.11. NR_CONVERGE_SPEED.....	60
3.12. IAA_HPF_FREQ.....	60
3.13. ANR_HANDLE.....	61
3.14. EQ_HANDLE.....	62
3.15. AGC_HANDLE.....	62
4. 错误码.....	64

1. 概述

1.1. 算法说明

APC (Audio Process Chain) 音频处理链路，是一个包含降噪、均衡器和自动增益控制的算法组合。APC 的主要目的是提高音频质量。APC 的内部的算法串接流程为 Anr → Eq/Hpf → Agc。透过降噪消除噪声提升 SNR，在根据客户所需曲线调整 EQ/HPF，最后输出透过 AGC 放大或压抑。

1.2. 关键词说明

- AGC

AGC(Automatic Gain Control), 自动增益控制，用于控制数字输出增益。

- EQ

EQ(Equalizer), 均衡器处理，用于对特定频段进行增益或衰减。

- ANR

ANR(Acoustic Noise Reduction), 降噪，用于去除环境中持续存在，频率固定的噪声。

- HPF

HPF(High-Pass Filtering), 高通滤波

1.3. 注意

为方便调试和确认算法效果，需要用户应用自行实现替换算法参数和抓取音频数据的逻辑。

2. API 参考

2.1. 功能模块 API

API 名	功能
IaaApc_GetBufferSize	获取 Apc 算法运行所需要的内存大小
IaaApc_Init	初始化 Apc 算法
IaaApc_Config	配置 Apc 算法
IaaApc_SetLowFreqCompressionRatioCurve	配置 Apc 中 Agc 算法分频段压缩率曲线(低频)
IaaApc_SetMidFreqCompressionRatioCurve	配置 Apc 中 Agc 算法分频段压缩率曲线(中频)
IaaApc_SetHighFreqCompressionRatioCurve	配置 Apc 中 Agc 算法分频段压缩率曲线(高频)
IaaApc_SetAgcFreqBand	配置 Apc 中 Agc 算法分频段之频率
IaaApc_GetNrResult	获取 Apc 算法 Anr 的输出数据
IaaApc_GetNrEqResult	获取 Apc 算法 Anr 和 Eq 的输出数据
IaaApc_Run	Apc 算法处理

API 名	功能
IaaApc_Free	释放 Apc 算法资源
IaaApc_Reset	重新初始化 Apc 算法
IaaApc_GetConfig	获取 Apc 算法当前的配置参数
IaaApc_ApplyDigitalGain	对输入数据做增益或衰减
IaaAnr_GetBufferSize	获取 Anr 算法运行所需要的内存大小
IaaAnr_Init	初始化 Anr 算法
IaaAnr_Config	配置 Anr 算法
IaaAnr_Run	Anr 算法处理
IaaAnr_Free	释放 Anr 算法资源
IaaAnr_Reset	重新初始化 Anr 算法
IaaAnr_GetConfig	获取 Anr 算法当前的配置参数
IaaEq_GetBufferSize	获取 Eq 算法运行所需要的内存大小
IaaEq_Init	初始化 Eq 算法
IaaEq_Config	配置 Eq 算法
IaaEq_Run	Eq 算法处理
IaaEq_Free	释放 Eq 算法资源
IaaEq_Reset	重新初始化 Eq 算法
IaaEq_GetConfig	获取 Eq 算法当前的配置参数
IaaAgc_GetBufferSize	获取 Agc 算法运行所需要的内存大小
IaaAgc_Init	初始化 Agc 算法
IaaAgc_Config	配置 Agc 算法
IaaAgc_Run	Agc 算法处理
IaaAgc_Free	释放 Agc 算法资源
IaaAgc_Reset	重新初始化 Agc 算法
IaaAgc_GetConfig	获取当前 Agc 算法的配置参数
IaaAgc_SetLowFreqCompressionRatioCurve	配置 Agc 算法分频段压缩率曲线(低频)
IaaAgc_SetMidFreqCompressionRatioCurve	配置 Agc 算法分频段压缩率曲线(中频)
IaaAgc_SetHighFreqCompressionRatioCurve	配置 Agc 算法分频段压缩率曲线(高频)
IaaAgc_SetAgcFreqBand	配置 Agc 算法分频段之频率

2.2. IaaApc_GetBufferSize

➤ 功能

获取 Apc 算法运行所需要的内存大小。

➤ 语法

```
unsigned int IaaApc_GetBufferSize(AudioApcBufferConfig *apc_switch);
```

➤ 形参

参数名称	描述	输入/输出
apc_switch	配置 Apc 算法使能设置的结构体指针	输入

➤ 返回值

返回值为 Apc 算法运行所需要的内存大小

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

该接口仅返回需要的内存大小，申请和释放内存的动作需应用来处理。

➤ 举例

请参考 [IaaApc_Run](#) 举例部分。

2.3. IaaApc_Init

➤ 功能

初始化 Apc 算法。

➤ 语法

```
APC_HANDLE IaaApc_Init(char* const working_buffer_address, AudioProcessInit  
*audio_process_init, AudioApcBufferConfig *apc_switch);
```

➤ 形参

参数名称	描述	输入/输出
working_buffer_address	Apc 算法所使用的内存地址	输入

audio_process_init	Apc 算法的初始化结构体指针	输入
apc_switch	Apc 算法使能结构体指针	输入

➤ 返回值

返回值	结果
非 NULL	成功
NULL	失败

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

- Apc 算法中 ANR/AGC/EQ 算法支持 8K/16K/48K 采样率, HPF 仅支援 8K/16K 采样率。

➤ 举例

请参考 [IaaApc_Run](#) 举例部分。

2.4. IaaApc_Config

➤ 功能

配置 Apc 算法。

➤ 语法

```
ALGO_APC_RET IaaApc_Config(APC\_HANDLE handle,
    AudioAnrConfig *anr_config,
    AudioEqConfig *eq_config,
    AudioHpfConfig *hpf_config,
    AudioVadConfig *vad_config,
    AudioDereverbConfig *dereverb_config,
    AudioAgcConfig *agc_config);
```

➤ 形参

参数名称	描述	输入/输出
handle	Apc 算法 handle	输入

anr_config	配置 Anr 算法的结构体指针	输入
eq_config	配置 Eq 算法的结构体指针	输入
hpf_config	配置 Hpf 算法的结构体指针	输入
vad_config	配置 Vad 算法的结构体指针（已废弃）	输入
dereverb_config	配置 Dereverb 算法的结构体指针（已废弃）	输入
agc_config	配置 Agc 算法的结构体指针	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败，参考 错误码

➤ 依赖

- 头文件：AudioProcess.h
- 库文件：libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaApc_Run](#) 举例部分。

2.5. IaaApc_SetLowFreqCompressionRatioCurve

➤ 功能

配置 Apc 中 Agc 算法分频段压缩率曲线(低频)。

➤ 语法

```
ALGO_APC_RET IaaApc_SetLowFreqCompressionRatioCurve(APC\_HANDLE handle,
int* CompressRatioInput, int* CompressRatioOutput);
```

➤ 形参

参数名称	描述	输入/输出
handle	Apc 算法 handle	输入
CompressRatioInput	压缩率输入，请参考 agc_config 中的	输入

	compression ratio	
CompressRatioOutput	压缩率输出, 请参考 agc_config 中的 compression ratio	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败, 参考 错误码

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaApc_Run](#) 举例部分。

2.6. IaaApc_SetMidFreqCompressionRatioCurve

➤ 功能

配置 Apc 中 Agc 算法分频段压缩率曲线(中频)。

➤ 语法

```
ALGO_APC_RET IaaApc_SetMidFreqCompressionRatioCurve(APC\_HANDLE handle,
int* CompressRatioInput, int* CompressRatioOutput);
```

➤ 形参

参数名称	描述	输入/输出
handle	Apc 算法 handle	输入
CompressRatioInput	压缩率输入, 请参考 agc_config 中的 compression ratio	输入
CompressRatioOutput	压缩率输出, 请参考 agc_config 中的 compression ratio	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败，参考 错误码

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaApc_Run](#) 举例部分。

2.7. IaaApc_SetHighFreqCompressionRatioCurve

➤ 功能

配置 Apc 中 Agc 算法分频段压缩率曲线(高频)。

➤ 语法

```
ALGO_APC_RET IaaApc_SetHighFreqCompressionRatioCurve(APC\_HANDLE handle,
int* CompressRatioInput, int* CompressRatioOutput);
```

➤ 形参

参数名称	描述	输入/输出
handle	Apc 算法 handle	输入
CompressRatioInput	压缩率输入，请参考 agc_config 中的 compression ratio	输入
CompressRatioOutput	压缩率输出，请参考 agc_config 中的 compression ratio	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败，参考 错误码

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaApc_Run](#) 举例部分。

2.8. IaaApc_SetAgcFreqBand

➤ 功能

配置 Apc 中 Agc 算法分频段之频率。

➤ 语法

```
ALGO_APC_RET IaaApc_SetAgcFreqBand(APC\_HANDLE handle,
                                     int* frequency_band);
```

➤ 形参

参数名称	描述	输入/输出
handle	Apc 算法 handle	输入
frequency_band	频段数组, 包含低频段上限频率, 中频段上限频率, 高频段上限频率。 举例: frequency_band 若设定为 {3000,4000,8000} 则低频频段为 0~3000Hz, 中频为 3000~4000Hz, 高频为 4000~8000Hz, 须注意最高频率不能超过采样率的一半。	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败, 参考 错误码

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaApc_Run](#) 举例部分。

2.9. IaaApc_GetNrResult

➤ 功能

获取 Apc 算法中 Anr 的处理结果。

➤ 语法

```
ALGO_APC_RET IaaApc_GetNrResult(APC\_HANDLE handle, short* nr_audio_out);
```

➤ 形参

参数名称	描述	输入/输出
handle	Apc 算法 handle	输入
nr_audio_out	Anr 处理输出数据指针	输出

➤ 返回值

返回值	结果
0	成功
非 0	失败, 参考 错误码

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

- 必须使能 Apc 算法中的 Anr, 才能调用此接口获取数据。

➤ 举例

请参考 [IaaApc_Run](#) 举例部分。

2.10. IaaApc_GetNrEqResult

➤ 功能

获取 Apc 算法中 Anr 和 Eq 的处理结果

➤ 语法

```
ALGO_APC_RET IaaApc_GetNrEqResult(APC\_HANDLE handle, short* nr_eq_audio_out);
```

➤ 形参

参数名称	描述	输入/输出
handle	Aec 算法 handle	输入
nr_eq_audio_out	Anr 和 Eq 处理输出数据指针	输出

➤ 返回值

返回值	结果
0	成功
非 0	失败，参考 错误码

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

- 必须使能 Apc 算法中的 Anr 和 Eq，才能调用此接口获取数据。

➤ 举例

请参考 [IaaApc_Run](#) 举例部分。

2.11. IaaApc_GetVadOut

➤ 功能

获取 Apc 算法中 Anr 计算的语音机率

➤ 语法

```
ALGO_APC_RET IaaApc_GetVadOut(APC\_HANDLE handle, int* speech_probability);
```

➤ 形参

参数名称	描述	输入/输出
handle	Aec 算法 handle	输入
speech_probability	Anr 处理输出语音机率数据指针 范围[0,100]	输出

➤ 返回值

返回值	结果
0	成功
非 0	失败，参考 错误码

➤ 依赖

- 头文件：AudioProcess.h
- 库文件：libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

- 必须使能 Apc 算法中的 Anr 和 Eq，才能调用此接口获取数据。

➤ 举例

请参考 [IaaApc_Run](#) 举例部分。

2.12. IaaApc_Run

➤ 功能

Apc 算法处理

➤ 语法

```
ALGO_APC_RET IaaApc_Run(APC\_HANDLE handle, short* pss_audio_in);
```

➤ 形参

参数名称	描述	输入/输出
handle	Apc 算法 handle	输入

pss_audio_in	输入数据指针	输入/输出
--------------	--------	-------

➤ 返回值

返回值	结果
0	成功
非 0	失败，参考 错误码

➤ 依赖

- 头文件: `AudioProcess.h`
- 库文件: `libAPC_LINUX.so/ libAPC_LINUX.a`

※ 注意

- 数据量必须和调用 `IaaApc_Init` 时设定的 `point_number`（一次 `IaaApc_Run` 所需要的采样点数）相对应，处理后的数据会回写到 `pss_audio_in` 所指向的内存中。

➤ 举例

[illegible]

```

-5, -5, -5, -5, -5, -5, -5, -5, -5, -5, -5, -5, -5, -5, -5, -5, -5 };
short compression_ratio_input[_AGC_CR_NUM] = { -65, -55, -48, -25, -18, -12, 0 };
short compression_ratio_output[_AGC_CR_NUM] = { -65, -50, -40, -20, -10, -5, -3 };

int i;
for (i = 0; i<30; i++)
{
    eq_table[i] = -20;
}

AudioApcBufferConfig apc_switch;
apc_switch.anr_enable = 1;
apc_switch.eq_enable = 0;
apc_switch.dr_enable = 0;
apc_switch.vad_enable = 0;
apc_switch.agc_enable = 1;

int buffersize = IaaApc_GetBufferSize(&apc_switch);
#if USE_MALLOC
char *working_buf_ptr = (char*)malloc(buffersize);
#else
char working_buf_ptr[512 * 100 * 2];
#endif

FILE * fin, *fout;
ALGO_APC_RET ret1;
AudioProcessInit apc_init;
AudioAnrConfig anr_config;
AudioEqConfig eq_config;
AudioHpfConfig hpf_config;
AudioAgcConfig agc_config;
APC_HANDLE handle;
apc_init.point_number = 128;
apc_init.channel = 1;
apc_init.sample_rate = IAA_APC_SAMPLE_RATE_16000;

handle = IaaApc_Init((char *)working_buf_ptr, &apc_init, &apc_switch);
if (handle == NULL)
{
    printf("APC init error\r\n");
    return -1;
}

```

```

}
else
{
    printf("APC init succeed\r\n");
}
/*****ANR Config*****/
anr_config.anr_enable = apc_switch.anr_enable;
anr_config.user_mode = 2;
memcpy(anr_config.anr_intensity_band, intensity_band, 6 * sizeof(int));
memcpy(anr_config.anr_intensity, intensity, 7 * sizeof(int));
anr_config.anr_smooth_level = 10;
anr_config.anr_converge_speed = 2;
/*****EQ Config*****/
eq_config.eq_enable = 0;
eq_config.user_mode = 1;
memcpy(eq_config.eq_gain_db, eq_table, _EQ_BAND_NUM*sizeof(short));
/*****HPF Config*****/
hpf_config.hpf_enable = apc_switch.eq_enable;
hpf_config.user_mode = 1;
hpf_config.cutoff_frequency = AUDIO_HPF_FREQ_500;
/*****AGC Config*****/
agc_config.agc_enable = apc_switch.agc_enable;
agc_config.user_mode = 2;
agc_config.gain_info.gain_max = 60;
agc_config.gain_info.gain_min = -5;
agc_config.gain_info.gain_init = 0;
agc_config.gain_step = 3;
agc_config.drop_gain_max = 50;
agc_config.attack_time = 1;
agc_config.release_time = 1;
agc_config.noise_gate_db = -50;
memcpy(agc_config.compression_ratio_input, compression_ratio_input,
_AGC_CR_NUM*sizeof(short));
memcpy(agc_config.compression_ratio_output, compression_ratio_output,
_AGC_CR_NUM*sizeof(short));
agc_config.noise_gate_attenuation_db = 0;
agc_config.drop_gain_threshold = -5;

int freqBand[_AGC_FREQ_BAND_NUM] = { 3000, 6000, 8000 };
int compressionRatioArrayLowInput[_AGC_CR_NUM] = { -80, -60, -40, -20, 0, 0, 0 };
int compressionRatioArrayLowOutput[_AGC_CR_NUM] = { -5, -5, -5, -5, -5, -5, -5 };
int compressionRatioArrayMidInput[_AGC_CR_NUM] = { -80, -60, -40, -20, 0, 0, 0 };
int compressionRatioArrayMidOutput[_AGC_CR_NUM] = { -80, -60, -40, -20, 0, 0, 0 };

```

```
int compressionRatioArrayHighInput[_AGC_CR_NUM] = { -80, -60, -40, -20, 0, 0, 0 };
int compressionRatioArrayHighOutput[_AGC_CR_NUM] = { -80, -60, -40, -20, 0, 0, 0 };

AudioProcessInit apc_init_cpy;
AudioAnrConfig anr_config_cpy;
AudioEqConfig eq_config_cpy;
AudioHpfConfig hpf_config_cpy;
AudioAgcConfig agc_config_cpy;

if (IaaApc_Config(handle, &anr_config, &eq_config, &hpf_config, NULL, NULL, &agc_config)
== -1)
{
    return -1;
}

IaaApc_SetAgcFreqBand(handle, freqBand);
IaaApc_SetLowFreqCompressionRatioCurve(handle, compressionRatioArrayLowInput,
compressionRatioArrayLowOutput);
IaaApc_SetMidFreqCompressionRatioCurve(handle, compressionRatioArrayMidInput,
compressionRatioArrayMidOutput);
IaaApc_SetHighFreqCompressionRatioCurve(handle, compressionRatioArrayHighInput,
compressionRatioArrayHighOutput);

IaaApc_GetConfig(handle, &apc_init_cpy, &anr_config_cpy, &eq_config_cpy, &hpf_config_cpy,
NULL, NULL, &agc_config_cpy);

sprintf(input_file, "%s", "../../code/sample/data/IEEE_Female_mono_16_kHz.wav");
sprintf(output_file, "%s", "../../code/sample/data/APC_out_16K.wav");

fin = fopen(input_file, "rb");
if (!fin)
{
    printf("the input file %s could not be open\n", input_file);
    return -1;
}

fout = fopen(output_file, "wb");
if (!fout)
{
    printf("the output file could not be open\n");
    return -1;
}
```

```

}

fread(input, sizeof(char), 44, fin); // read header 44 bytes
fwrite(input, sizeof(char), 44, fout); // write 44 bytes output

while (fread(input, sizeof(short), apc_init.point_number*apc_init.channel, fin))
{
    counter++;
    // do one frame audio process -----
    ret1 = IaaApc_Run(handle, input);
    //      IaaApc_GetNrEqResult(handle, input);
    //      IaaApc_GetNrResult(handle, input);
    IaaApc_GetVadOut(handle, &speech_prob);
    if (counter % 1000 == 999)
    {
        printf("counter = %d\n", counter);
        printf("current time = %f\n", (float)counter*apc_init.point_number /
apc_init.sample_rate);
    }

    if (ret1 != ALGO_APC_RET_SUCCESS)
    {
        printf("Error occurred in AudioProcess\n");
        break;
    }

    fwrite(input, sizeof(short), apc_init.point_number*apc_init.channel, fout);
}
IaaApc_Free(handle);
fclose(fin);
fclose(fout);

printf("Done\n");
return 0;
}

```

2.13. IaaApc_Free

➤ 功能

释放 Apc 算法资源

➤ 语法

ALGO_APC_RET IaaApc_Free([APC_HANDLE](#) handle);

➤ 形参

参数名称	描述	输入/输出
handle	Apc 算法 handle	输入

➤ 返回值

无。

➤ 依赖

- 头文件: [AudioProcess.h](#)
- 库文件: [libAPC_LINUX.so](#)/ [libAPC_LINUX.a](#)

※ 注意

无。

➤ 举例

请参考 [IaaApc_Run](#) 举例部分。

2.14. IaaApc_Reset

➤ 功能

重新初始化 Apc 算法

➤ 语法

[APC_HANDLE](#) IaaApc_Reset(char* working_buffer_address,[AudioProcessInit](#) *audio_process_init,
[AudioApcBufferConfig](#) *apc_switch);

➤ 形参

参数名称	描述	输入/输出
working_buffer_address	Apc 算法运行所使用的内存地址	输入
audio_process_init	Apc 算法的初始化结构体指针	输入
apc_switch	配置 Apc 算法使能设置的结构体指针	输入

➤ 返回值

返回值	结果
-----	----

非 NULL	成功
NULL	失败

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

当重新初始化 Apc 算法且使用的功能选项与之前不同时，需要释放掉原来算法运行所使用的内存，重新调用 [IaaApc_GetBufferSize](#) 获取当前功能选项所需要的内存大小，重新申请相应大小的内存供 Apc 算法使用。

➤ 举例

无。

2.15. IaaApc_GetConfig

➤ 功能

获取 Apc 算法的当前配置参数

➤ 语法

```
ALGO_APC_RET IaaApc_GetConfig(APC\_HANDLE handle,
    AudioProcessInit *audio_process_init,
    AudioAnrConfig *anr_config,
    AudioEqConfig *eq_config,
    AudioHpfConfig *hpf_config,
    AudioVadConfig *vad_config,
    AudioDereverbConfig *dereverb_config,
    AudioAgcConfig *agc_config);
```

➤ 形参

参数名称	描述	输入/输出
handle	Apc 算法 handle	输入
audio_process_init	Apc 算法的初始化结构体指针	输出
anr_config	Apc 算法中当前 Anr 的配置参数	输出
eq_config	Apc 算法中当前 Eq 的配置参数	输出
hpf_config	Apc 算法中当前 Hpf 的配置参数	输出

vad_config	Apc 算法中当前 Vad 的配置参数（已废弃）	输出
dereverb_config	Apc 算法中当前 Dereverb 的配置参数（已废弃）	输出
agc_config	Apc 算法中当前 Agc 的配置参数	输出

➤ 返回值

返回值	结果
0	成功
非 0	失败，参考 错误码

➤ 依赖

- 头文件：AudioProcess.h
- 库文件：libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaApc_Run](#) 举例部分。

2.16. IaaApc_ApplyDigitalGain

➤ 功能

对输入数据做增益或衰减

➤ 语法

```
ALGO_APC_RET    IaaApc_ApplyDigitalGain(APC_HANDLE handle, short* pss_audio_in, float
gain_value      );
```

➤ 形参

参数名称	描述	输入/输出
handle	Apc 算法 handle	输入
pss_audio_in	输入数据指针	输入/输出
gain_value	增益/衰减值	输入

➤ 返回值

返回值	结果
0	成功

返回值	结果
非 0	失败，参考 错误码

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

2.17. IaaAnr_GetBufferSize

➤ 功能

获取 Anr 算法运行所需要的内存大小。

➤ 语法

```
unsigned int IaaAnr_GetBufferSize(void);
```

➤ 形参

参数名称	描述	输入/输出
------	----	-------

➤ 返回值

返回值为 Anr 算法运行所需要的内存大小

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

该接口仅返回需要的内存大小，申请和释放内存的动作需应用来处理。

➤ 举例

请参考 [IaaAnr_Run](#) 举例部分。

2.18. IaaAnr_Init

➤ 功能

初始化 Anr 算法。

➤ 语法

```
ANR_HANDLE IaaAnr_Init(char* working_buffer_address, AudioProcessInit *anr_init);
```

➤ 形参

参数名称	描述	输入/输出
working_buffer_address	Anr 算法所使用的内存地址	输入
anr_init	Anr 算法的初始化结构体指针	输入

➤ 返回值

返回值	结果
非 NULL	成功
NULL	失败

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

- ANR 算法支持 8K/16K/48K 采样率。

➤ 举例

请参考 [IaaAnr_Run](#) 举例部分。

2.19. IaaAnr_Config

➤ 功能

配置 Anr 算法。

➤ 语法

```
ALGO_APC_RET IaaAnr_Config(ANR_HANDLE handle, AudioAnrConfig *anr_config);
```

➤ 形参

参数名称	描述	输入/输出
handle	Anr 算法 handle	输入
anr_config	Anr 算法的配置结构体指针	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败，参考 错误码

➤ 依赖

- 头文件：AudioProcess.h
- 库文件：libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaAnr_Run](#) 举例部分。

2.20. IaaAnr_Run

➤ 功能

Anr 算法处理

➤ 语法

```
ALGO_APC_RET IaaAnr_Run(ANR\_HANDLE handle, short* pss_audio_in);
```

➤ 形参

参数名称	描述	输入/输出
handle	Anr 算法 handle	输入
pss_audio_in	输入数据指针	输入/输出

➤ 返回值

返回值	结果
0	成功
非 0	失败，参考 错误码

➤ 依赖

- 头文件：AudioProcess.h
- 库文件：libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

- 数据量必须和调用 IaaAnr_Init 时设定的 point_number（一次 IaaAnr_Run 所需要的采样点数）相对应，处理后的数据会回写到 pss_audio_in 所指向的内存中。

➤ 举例

```

1. #include <stdio.h>
2. #include <string.h>
3. #include <stdlib.h>
4. #include <sys/time.h>
5.
6. #include "AudioProcess.h"
7.
8. /* 0:Fixed input file 1:User input file */
9. #define IN_PARAMETER 1
10.
11. int main(int argc, char *argv[])
12. {
13.     short in_output[1024];
14.     unsigned int workingBufferSize;
15.     char *workingBuffer = NULL;
16.     ANR_HANDLE handle;
17.     AudioProcessInit anr_init, anr_get_init;
18.     AudioAnrConfig anr_config, anr_get_config;
19.     ALGO_APC_RET ret;
20.     int tempSize;
21.     FILE* fpIn; //input file
22.     FILE* fpOut; //output file
23.     char src_file[128] = {0};
24.     char dst_file[128] = {0};
25.     /*****User change section start*****/
26.     int intensity_band[6] = {3,24,40,64,80,128};
27.     int intensity[7] = {30,30,30,30,30,30,30};
28.     anr_init.point_number = 128;
29.     anr_init.channel = 1;
30.     anr_init.sample_rate = IAA_APC_SAMPLE_RATE_16000;
31.
32.     anr_config.anr_enable = 1;
33.     anr_config.user_mode = 2;
34.     anr_config.anr_smooth_level = 10;
35.     anr_config.anr_converge_speed = 0;
36.     /*****User change section end*****/
37.     memcpy(anr_config.anr_intensity_band, intensity_band, sizeof(intensity_band));

```

```

38. memcpy(anr_config.anr_intensity, intensity, sizeof(intensity));
39. //(1)IaaAnr_GetBufferSize
40. workingBufferSize = IaaAnr_GetBufferSize();
41. workingBuffer = (char *)malloc(workingBufferSize);
42. if(NULL == workingBuffer)
43. {
44.     printf("malloc workingBuffer failed !\n");
45.     return -1;
46. }
47. printf("malloc workingBuffer succeed !\n");
48. //(2)IaaAnr_Init
49. handle = IaaAnr_Init(workingBuffer, &anr_init);
50. if(NULL == handle)
51. {
52.     printf("IaaAnr_Init failed !\n");
53.     return -1;
54. }
55. printf("IaaAnr_Init succeed !\n");
56. //(3)IaaAnr_Config
57. ret = IaaAnr_Config(handle, &anr_config);
58. if(ret)
59. {
60.     printf("IaaAnr_Config failed !\n");
61.     return -1;
62. }
63. printf("IaaAnr_Config succeed !\n");
64. //(4)IaaAnr_GetConfig
65. ret = IaaAnr_GetConfig(handle, &anr_get_init, &anr_get_config);
66. if(ret)
67. {
68.     printf("IaaAnr_GetConfig failed !\n");
69.     return -1;
70. }
71. printf("IaaAnr_GetConfig succeed !\n");
72. printf("anr_get_config.user_mode = %d, ...\n", anr_get_config.user_mode);
73.
74. #if IN_PARAMETER
75. if(argc < 3)
76. {
77.     printf("Please enter the correct parameters!\n");
78.     return -1;
79. }
80. sscanf(argv[1], "%s", src_file);
81. sscanf(argv[2], "%s", dst_file);
82. #else
83. sprintf(src_file, "%s", "./APC_AFE_16K.wav");
84. if(argc < 2)
85. {
86.     printf("Please enter the correct parameters!\n");
87.     return -1;
88. }
89. sscanf(argv[1], "%s", dst_file);
90. #endif
91.
92. fpIn = fopen(src_file, "rb");
93. if(NULL == fpIn)
94. {
95.     printf("fopen in_file failed !\n");
96.     return -1;
97. }

```

```

98.  printf("fopen in_file success !\n");
99.  fpOut = fopen(dst_file, "wb");
100. if(NULL == fpOut)
101. {
102.     printf("fopen out_file failed !\n");
103.     return -1;
104. }
105. printf("fopen out_file success !\n");
106. #if 1
107.     fread(in_output, sizeof(char), 44, fpIn);
108.     fwrite(in_output, sizeof(char), 44, fpOut);
109. #endif
110.     tempSize = anr_init.point_number * anr_init.channel;
111.     while(tempSize == fread(in_output, sizeof(short), tempSize, fpIn))
112.     {
113.         //(5)IaaAnr_Run
114.         ret = IaaAnr_Run(handle, in_output);
115.         if(ret)
116.         {
117.             printf("IaaAnr_Run failed !\n");
118.             return -1;
119.         }
120.         fwrite(in_output, sizeof(short), tempSize, fpOut);
121.     }
122.     printf("Break:needBytes =%d \t nBytesRead = %d\n", anr_init.point_number * anr_init.channel, tempSize);
123.     //(6)IaaAnr_Free
124.     IaaAnr_Free(handle);
125.     free(workingBuffer);
126.     fclose(fpIn);
127.     fclose(fpOut);
128.     printf("APC_ANR end !\n");
129.
130.     return 0;
131. }

```

2.21. IaaAnr_Free

➤ 功能

释放 Anr 算法资源

➤ 语法

ALGO_APC_RET IaaAnr_Free([ANR_HANDLE](#) handle);

➤ 形参

参数名称	描述	输入/输出
handle	Anr 算法 handle	输入

➤ 返回值

无。

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaAnr_Run](#) 举例部分。

2.22. IaaAnr_Reset

➤ 功能

重新初始化 Anr 算法

➤ 语法

```
ANR_HANDLE IaaAnr_Reset(char* working_buffer_address, AudioProcessInit *anr_init);
```

➤ 形参

参数名称	描述	输入/输出
working_buffer_address	Anr 算法运行所使用的内存地址	输入
anr_init	Anr 算法的初始化结构体指针	输入

➤ 返回值

返回值	结果
非 NULL	成功
NULL	失败

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

无。

2.23. IaaAnr_GetConfig

➤ 功能

获取 Anr 算法的当前配置参数

➤ 语法

```
ALGO_APC_RET IaaAnr_GetConfig(ANR\_HANDLE handle, AudioProcessInit *anr_init, AudioAnrConfig *anr_config);
```

➤ 形参

参数名称	描述	输入/输出
handle	Anr 算法 handle	输入
anr_init	Anr 算法的初始化结构体指针	输出
anr_config	Anr 算法中当前的配置参数	输出

➤ 返回值

返回值	结果
0	成功
非 0	失败，参考 错误码

➤ 依赖

- 头文件: [AudioProcess.h](#)
- 库文件: [libAPC_LINUX.so](#)/ [libAPC_LINUX.a](#)

※ 注意

无。

➤ 举例

请参考 [IaaAnr_Run](#) 举例部分。

2.24. IaaEq_GetBufferSize

➤ 功能

获取 Eq 算法运行所需要的内存大小。

➤ 语法

```
unsigned int IaaEq_GetBufferSize(void);
```

➤ 形参

参数名称	描述	输入/输出
------	----	-------

➤ 返回值

返回值为 Eq 算法运行所需要的内存大小

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

该接口仅返回需要的内存大小，申请和释放内存的动作需应用来处理。

➤ 举例

请参考 [IaaEq_Run](#) 举例部分。

2.25. IaaEq_Init

➤ 功能

初始化 Eq 算法。

➤ 语法

```
EQ_HANDLE IaaEq_Init(char* working_buffer_address, AudioProcessInit *eq_init);
```

➤ 形参

参数名称	描述	输入/输出
working_buffer_address	Eq 算法所使用的内存地址	输入
eq_init	Eq 算法的初始化结构体指针	输入

➤ 返回值

返回值	结果
非 NULL	成功
NULL	失败

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

- Eq 算法支持 8K/16K/48K 采样率, Hpf 仅支援 8K/16K 采样率。

➤ 举例

请参考 [IaaEq_Run](#) 举例部分。

2.26. IaaEq_Config

➤ 功能

配置 Eq 算法。

➤ 语法

```
ALGO_APC_RET IaaEq_Config(EQ_HANDLE handle, AudioHpfConfig *hpf_config, AudioEqConfig *eq_config);
```

➤ 形参

参数名称	描述	输入/输出
handle	Eq 算法 handle	输入
hpf_config	Hpf 算法的配置结构体指针	输入
eq_config	Eq 算法的配置结构体指针	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败, 参考 错误码

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaEq_Run](#) 举例部分。

2.27. IaaEq_Run

➤ 功能

Eq 算法处理

➤ 语法

```
ALGO_APC_RET IaaEq_Run(EQ_HANDLE handle, short* pss_audio_in);
```

➤ 形参

参数名称	描述	输入/输出
handle	Eq 算法 handle	输入
pss_audio_in	输入数据指针	输入/输出

➤ 返回值

返回值	结果
0	成功
非 0	失败, 参考 错误码

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

- 数据量必须和调用 [IaaEq_Init](#) 时设定的 `point_number` (一次 [IaaEq_Run](#) 所需要的采样点数) 相对应, 处理后的数据会回写到 `pss_audio_in` 所指向的内存中。

➤ 举例

```

1. #include <stdio.h>
2. #include <string.h>
3. #include <stdlib.h>
4. #include <sys/time.h>
5.
6. #include "AudioProcess.h"
7.
8. /* 0:Fixed input file 1:User input file */
9. #define IN_PARAMETER 1
10.
11. int main(int argc, char *argv[])
12. {
13.     short in_output[1024];
14.     unsigned int workingBufferSize;
15.     char *workingBuffer = NULL;
16.     EQ_HANDLE handle;
17.     AudioProcessInit eq_init, eq_get_init;
18.     AudioHpfConfig hpf_config, hpf_get_config;
19.     AudioEqConfig eq_config, eq_get_config;
20.     ALGO_APC_RET ret;
21.     int tempSize;
22.
23.     FILE* fpIn; //input file
24.     FILE* fpOut; //output file
25.     char src_file[128] = {0};
26.     char dst_file[128] = {0};
27.     /*****User change section start*****/
28.     short eq_table[129];
29.     memset(eq_table, 0, sizeof(eq_table));
30.     eq_init.point_number = 128;
31.     eq_init.channel = 1;
32.     eq_init.sample_rate = IAA_APC_SAMPLE_RATE_16000;
33.
34.     hpf_config.hpf_enable = 1;
35.     hpf_config.user_mode = 1;
36.     hpf_config.cutoff_frequency = AUDIO_HPF_FREQ_150;
37.
38.     eq_config.eq_enable = 1;
39.     eq_config.user_mode = 1;
40.     /*****User change section end*****/
41.     memcpy(eq_config.eq_gain_db, eq_table, sizeof(eq_table));
42.     //(1)IaaEq_GetBufferSize
43.     workingBufferSize = IaaEq_GetBufferSize();
44.     workingBuffer = (char *)malloc(workingBufferSize);
45.     if(NULL == workingBuffer)
46.     {
47.         printf("malloc workingBuffer failed !\n");
48.         return -1;
49.     }
50.     printf("malloc workingBuffer succeed !\n");
51.     //(2)IaaEq_Init
52.     handle = IaaEq_Init(workingBuffer, &eq_init);
53.     if(NULL == handle)
54.     {
55.         printf("IaaEq_Init failed !\n");
56.         return -1;

```

```

57. }
58. printf("IaaEq_Init succeed !\n");
59. //(3)IaaEq_Config
60. ret = IaaEq_Config(handle, &hpf_config, &eq_config);
61. if(ret)
62. {
63.     printf("IaaEq_Config failed !\n");
64.     return -1;
65. }
66. printf("IaaEq_Config succeed !\n");
67. //(4)IaaEq_GetConfig
68. ret = IaaEq_GetConfig(handle, &eq_get_init, &hpf_get_config, &eq_get_config);
69. if(ret)
70. {
71.     printf("IaaEq_GetConfig failed !\n");
72.     return -1;
73. }
74. printf("IaaEq_GetConfig succeed !\n");
75. printf("eq_get_config.user_mode = %d, ...\n", eq_get_config.user_mode);
76.
77. #if IN_PARAMETER
78. if(argc < 3)
79. {
80.     printf("Please enter the correct parameters!\n");
81.     return -1;
82. }
83. sscanf(argv[1], "%s", src_file);
84. sscanf(argv[2], "%s", dst_file);
85. #else
86. sprintf(src_file, "%s", "./APC_AFE_16K.wav");
87. if(argc < 2)
88. {
89.     printf("Please enter the correct parameters!\n");
90.     return -1;
91. }
92. sscanf(argv[1], "%s", dst_file);
93. #endif
94.
95. fpIn = fopen(src_file, "rb");
96. if(NULL == fpIn)
97. {
98.     printf("fopen in_file failed !\n");
99.     return -1;
100. }
101. printf("fopen in_file success !\n");
102. fpOut = fopen(dst_file, "wb");
103. if(NULL == fpOut)
104. {
105.     printf("fopen out_file failed !\n");
106.     return -1;
107. }
108. printf("fopen out_file success !\n");
109. #if 1
110. fread(in_output, sizeof(char), 44, fpIn);
111. fwrite(in_output, sizeof(char), 44, fpOut);
112. #endif
113. tempSize = eq_init.point_number * eq_init.channel;
114. while(tempSize == fread(in_output, sizeof(short), tempSize, fpIn))
115. {
116.     //(5)IaaEq_Run

```

```

117.     ret = IaaEq_Run(handle, in_output);
118.     if(ret)
119.     {
120.         printf("IaaEq_Run failed !\n");
121.         return -1;
122.     }
123.     fwrite(in_output, sizeof(short), tempSize, fpOut);
124. }
125. printf("Break:needBytes =%d \t nBytesRead = %d\n", eq_init.point_number * eq_init.channel, tempSize);
126. //(6)IaaEq_Free
127. IaaEq_Free(handle);
128. free(workingBuffer);
129. fclose(fpIn);
130. fclose(fpOut);
131. printf("APC_EQ end !\n");
132.
133. return 0;
134.}

```

2.28. IaaEq_Free

➤ 功能

释放 Eq 算法资源

➤ 语法

ALGO_APC_RET IaaEq_Free(EQ_HANDLE handle);

➤ 形参

参数名称	描述	输入/输出
handle	Eq 算法 handle	输入

➤ 返回值

无。

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaEq_Run](#) 举例部分。

2.29. IaaEq_Reset

➤ 功能

重新初始化 Eq 算法

➤ 语法

```
EQ_HANDLE IaaEq_Reset(char* working_buffer_address, AudioProcessInit *eq_init);
```

➤ 形参

参数名称	描述	输入/输出
working_buffer_address	Eq 算法运行所使用的内存地址	输入
eq_init	Eq 算法的初始化结构体指针	输入

➤ 返回值

返回值	结果
非 NULL	成功
NULL	失败

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

无。

2.30. IaaEq_GetConfig

➤ 功能

获取 Eq 算法的当前配置参数

➤ 语法

```
ALGO_APC_RET IaaEq_GetConfig(EQ_HANDLE handle, AudioProcessInit *eq_init, AudioHpfConfig *hpf_config, AudioEqConfig *eq_config);
```

➤ 形参

参数名称	描述	输入/输出
handle	Eq 算法 handle	输入
eq_init	Eq 算法的初始化结构体指针	输出
hpf_config	Hpf 算法中当前的配置参数	输出
eq_config	Eq 算法中当前的配置参数	输出

➤ 返回值

返回值	结果
0	成功
非 0	失败，参考 错误码

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaEq_Run](#) 举例部分。

2.31. IaaAgc_GetBufferSize

➤ 功能

获取 Agc 算法运行所需要的内存大小。

➤ 语法

```
unsigned int IaaAgc_GetBufferSize(void);
```

➤ 形参

参数名称	描述	输入/输出
------	----	-------

➤ 返回值

返回值为 Agc 算法运行所需要的内存大小

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

该接口仅返回需要的内存大小，申请和释放内存的动作需应用来处理。

➤ 举例

请参考 [IaaAgc_Run](#) 举例部分。

2.32. IaaAgc_Init

➤ 功能

初始化 Agc 算法。

➤ 语法

```
AGC_HANDLE IaaAgc_Init(char* working_buffer_address, AudioProcessInit *agc_init);
```

➤ 形参

参数名称	描述	输入/输出
working_buffer_address	Agc 算法所使用的内存地址	输入
agc_init	Agc 算法的初始化结构体指针	输入

➤ 返回值

返回值	结果
非 NULL	成功
NULL	失败

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

- Agc 算法支持 8K/16K/48K 采样率。

➤ 举例

请参考 [IaaAgc_Run](#) 举例部分。

2.33. IaaAgc_Config

➤ 功能

配置 Agc 算法。

➤ 语法

```
ALGO_APC_RET IaaAgc_Config(AGC\_HANDLE handle, AudioAgcConfig *agc_config);
```

➤ 形参

参数名称	描述	输入/输出
handle	Agc 算法 handle	输入
agc_config	Agc 算法的配置结构体指针	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败，参考 错误码

➤ 依赖

- 头文件: `AudioProcess.h`
- 库文件: `libAPC_LINUX.so/ libAPC_LINUX.a`

※ 注意

无。

➤ 举例

请参考 [IaaAgc_Run](#) 举例部分。

2.34. IaaAgc_SetLowFreqCompressionRatioCurve

➤ 功能

配置 Agc 算法分频段压缩率曲线(低频)。

➤ 语法

```
ALGO_AGC_RET IaaAgc_SetLowFreqCompressionRatioCurve(AGC\_HANDLE handle,
int* CompressRatioInput, int* CompressRatioOutput);
```

➤ 形参

参数名称	描述	输入/输出
handle	Agc 算法 handle	输入
CompressRatioInput	压缩率输入, 请参考 agc_config 中的 compression ratio	输入
CompressRatioOutput	压缩率输出, 请参考 agc_config 中的 compression ratio	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败, 参考 错误码

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaAgc_Run](#) 举例部分。

2.35. IaaAgc_SetMidFreqCompressionRatioCurve

➤ 功能

配置 Agc 算法分频段压缩率曲线(中频)。

➤ 语法

ALGO_AGC_RET IaaAgc_SetMidFreqCompressionRatioCurve([AGC_HANDLE](#) handle,
int* CompressRatioInput, int* CompressRatioOutput);

➤ 形参

参数名称	描述	输入/输出
handle	Agc 算法 handle	输入
CompressRatioInput	压缩率输入, 请参考 agc_config 中的 compression ratio	输入
CompressRatioOutput	压缩率输出, 请参考 agc_config 中的 compression ratio	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败, 参考 错误码

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaAgc_Run](#) 举例部分。

2.36. IaaAgc_SetHighFreqCompressionRatioCurve

➤ 功能

配置 Agc 算法分频段压缩率曲线(高频)。

➤ 语法

ALGO_AGC_RET IaaAgc_SetHighFreqCompressionRatioCurve([AGC_HANDLE](#) handle,
int* CompressRatioInput, int* CompressRatioOutput);

➤ 形参

参数名称	描述	输入/输出
handle	Agc 算法 handle	输入
CompressRatioInput	压缩率输入, 请参考 agc_config 中的 compression ratio	输入
CompressRatioOutput	压缩率输出, 请参考 agc_config 中的 compression ratio	输入

➤ 返回值

返回值	结果
0	成功
非 0	失败, 参考 错误码

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaAgc_Run](#) 举例部分。

2.37. IaaAgc_SetAgcFreqBand

➤ 功能

配置 Agc 算法分频段之频率。

➤ 语法

```
ALGO_AGC_RET IaaAgc_SetAgcFreqBand(AGC\_HANDLE handle,
                                     int* frequency_band);
```

➤ 形参

参数名称	描述	输入/输出
handle	Agc 算法 handle	输入
frequency_band	频段数组, 包含低频段上限频率, 中频段上限频率, 高频段上限频率。 举例: frequency_band 若设定为	输入

	{3000,4000,8000} 则低频频段为 0~3000Hz，中频为 3000~4000 Hz，高频为 4000~8000 Hz，须注意最高频率不能超过采样率的一半。	
--	--	--

➤ 返回值

返回值	结果
0	成功
非 0	失败，参考 错误码

➤ 依赖

- 头文件：AudioProcess.h
- 库文件：libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaAgc_Run](#) 举例部分。

2.38. IaaAgc_Run

➤ 功能

Agc 算法处理

➤ 语法

```
ALGO_APC_RET IaaAgc_Run(AGC\_HANDLE handle, short* pss_audio_in);
```

➤ 形参

参数名称	描述	输入/输出
handle	Agc 算法 handle	输入
pss_audio_in	输入数据指针	输入/输出

➤ 返回值

返回值	结果
0	成功

返回值	结果
非 0	失败，参考 错误码

➤ 依赖

- 头文件：AudioProcess.h
- 库文件：libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

- 数据量必须和调用 IaaAgc_Init 时设定的 point_number（一次 IaaAgc_Run 所需要的采样点数）相对应，处理后的数据会回写到 pss_audio_in 所指向的内存中。

➤ 举例

```

1. #include <stdio.h>
2. #include <string.h>
3. #include <stdlib.h>
4. #include <sys/time.h>
5.
6. #include "AudioProcess.h"
7.
8. /* 0:Fixed input file 1:User input file */
9. #define IN_PARAMETER 1
10.
11. int main(int argc, char *argv[])
12. {
13.     short in_output[1024];
14.     unsigned int workingBufferSize;
15.     char *workingBuffer = NULL;
16.     AGC_HANDLE handle;
17.     AudioProcessInit agc_init, agc_get_init;
18.     AudioAgcConfig agc_config, agc_get_config;
19.     ALGO_APC_RET ret;
20.     int tempSize;
21.     FILE* fpIn; //input file
22.     FILE* fpOut; //output file
23.     char src_file[128] = {0};
24.     char dst_file[128] = {0};
25.     /******User change section start*****/
26.     short compression_ratio_input[7] = {-65,-55,-48,-25,-18,-12,0};
27.     short compression_ratio_output[7] = {-65,-50,-27,-12,-1,-1,-1};
28.     int freqBand[7] = {1000,6000,8000};
29.     int compressionRatioArrayLowInput[7] = {-65,-55,-48,-25,-18,-12,0};
30.     int compressionRatioArrayLowOutput[7] = {-65,-50,-27,-12,-1,-1,-1};
31.     int compressionRatioArrayMidInput[7] = {-65,-55,-48,-25,-18,-12,0};
32.     int compressionRatioArrayMidOutput[7] = {-65,-50,-27,-12,-1,-1,-1};
33.     int compressionRatioArrayHighInput[7] = {-65,-55,-48,-25,-18,-12,0};
34.     int compressionRatioArrayHighOutput[7] = {-65,-50,-27,-12,-1,-1,-1};
35.     agc_init.point_number = 128;
36.     agc_init.channel = 1;
37.     agc_init.sample_rate = IAA_APC_SAMPLE_RATE_16000;
38.
39.     agc_config.agc_enable = 1;

```

```

40.  agc_config.user_mode = 2;
41.  agc_config.gain_info.gain_max = 40;
42.  agc_config.gain_info.gain_min = -10;
43.  agc_config.gain_info.gain_init = 12;
44.  agc_config.drop_gain_max = 36;
45.  agc_config.drop_gain_step = 1;
46.  agc_config.attack_time = 1;
47.  agc_config.release_time = 1;
48.  agc_config.noise_gate_db = -80;
49.  agc_config.noise_gate_attenuation_db = 0;
50.  agc_config.drop_gain_threshold = -5;
51.  /*****User change section end*****/
52.  memcpy(agc_config.compression_ratio_input, compression_ratio_input, sizeof(compression_ratio_input));
53.  memcpy(agc_config.compression_ratio_output, compression_ratio_output, sizeof(compression_ratio_output));
54.  //(1)IaaAgc_GetBufferSize
55.  workingBufferSize = IaaAgc_GetBufferSize();
56.  workingBuffer = (char *)malloc(workingBufferSize);
57.  if(NULL == workingBuffer)
58.  {
59.      printf("malloc workingBuffer failed !\n");
60.      return -1;
61.  }
62.  printf("malloc workingBuffer succeed !\n");
63.  //(2)IaaAgc_Init
64.  handle = IaaAgc_Init(workingBuffer, &agc_init);
65.  if(NULL == handle)
66.  {
67.      printf("IaaAgc_Init failed !\n");
68.      return -1;
69.  }
70.  printf("IaaAgc_Init succeed !\n");
71.  //(3)IaaAgc_Config
72.  ret = IaaAgc_Config(handle, &agc_config);
73.  if(ret)
74.  {
75.      printf("IaaAgc_Config failed !\n");
76.      return -1;
77.  }
78.  printf("IaaAgc_Config succeed !\n");
79.
80.  IaaAgc_SetAgcFreqBand(handle, freqBand);
81.  IaaAgc_SetLowFreqCompressionRatioCurve(handle,compressionRatioArrayLowInput,
compressionRatioArrayLowOutput);
82.  IaaAgc_SetMidFreqCompressionRatioCurve(handle,compressionRatioArrayMidInput,
compressionRatioArrayMidOutput);
83.  IaaAgc_SetHighFreqCompressionRatioCurve(handle,compressionRatioArrayHighInput,
compressionRatioArrayHighOutput);
84.  //(4)IaaAgc_GetConfig
85.  ret = IaaAgc_GetConfig(handle, &agc_get_init, &agc_get_config);
86.  if(ret)
87.  {
88.      printf("IaaAgc_GetConfig failed !\n");
89.      return -1;
90.  }
91.  printf("IaaAgc_GetConfig succeed !\n");
92.  printf("agc_get_config.user_mode = %d, ...\n", agc_get_config.user_mode);
93.
94.  #if IN_PARAMETER
95.  if(argc < 3)
96.  {

```



```

97.     printf("Please enter the correct parameters!\n");
98.     return -1;
99. }
100. sscanf(argv[1], "%s", src_file);
101. sscanf(argv[2], "%s", dst_file);
102. #else
103.     sprintf(src_file, "%s", "./APC_AFE_16K.wav");
104.     if(argc < 2)
105.     {
106.         printf("Please enter the correct parameters!\n");
107.         return -1;
108.     }
109.     sscanf(argv[1], "%s", dst_file);
110. #endif
111.
112.     fpIn = fopen(src_file, "rb");
113.     if(NULL == fpIn)
114.     {
115.         printf("fopen in_file failed !\n");
116.         return -1;
117.     }
118.     printf("fopen in_file success !\n");
119.     fpOut = fopen(dst_file, "wb");
120.     if(NULL == fpOut)
121.     {
122.         printf("fopen out_file failed !\n");
123.         return -1;
124.     }
125.     printf("fopen out_file success !\n");
126. #if 1
127.     fread(in_output, sizeof(char), 44, fpIn);
128.     fwrite(in_output, sizeof(char), 44, fpOut);
129. #endif
130.     tempSize = agc_init.point_number * agc_init.channel;
131.     while(tempSize == fread(in_output, sizeof(short), tempSize, fpIn))
132.     {
133.         //(5)IaaAgc_Run
134.         ret = IaaAgc_Run(handle, in_output);
135.         if(ret)
136.         {
137.             printf("IaaAnr_Run failed !\n");
138.             return -1;
139.         }
140.         fwrite(in_output, sizeof(short), tempSize, fpOut);
141.     }
142.     printf("Break:needBytes =%d \t nBytesRead = %d\n", agc_init.point_number * agc_init.channel, tempSize);
143.     //(6)IaaAgc_Free
144.     IaaAgc_Free(handle);
145.     free(workingBuffer);
146.     fclose(fpIn);
147.     fclose(fpOut);
148.     printf("APC_AGC end !\n");
149.
150.     return 0;
151. }

```

2.39. IaaAgc_Free

➤ 功能

释放 Agc 算法资源

➤ 语法

```
ALGO_APC_RET IaaAgc_Free(AGC\_HANDLE handle);
```

➤ 形参

参数名称	描述	输入/输出
handle	Agc 算法 handle	输入

➤ 返回值

无。

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaAgc_Run](#) 举例部分。

2.40. IaaAgc_Reset

➤ 功能

重新初始化 Agc 算法

➤ 语法

```
AGC\_HANDLE IaaAgc_Reset(char* working_buffer_address, AudioProcessInit *agc_init);
```

➤ 形参

参数名称	描述	输入/输出
working_buffer_address	Agc 算法运行所使用的内存地址	输入
agc_init	Agc 算法的初始化结构体指针	输入

➤ 返回值

返回值	结果
非 NULL	成功
NULL	失败

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

无。

2.41. IaaAgc_GetConfig

➤ 功能

获取 Agc 算法的当前配置参数

➤ 语法

```
ALGO_APC_RET IaaAgc_GetConfig(AGC_HANDLE handle, AudioProcessInit *agc_init, AudioAgcConfig *agc_config);
```

➤ 形参

参数名称	描述	输入/输出
handle	Agc 算法 handle	输入
agc_init	Agc 算法的初始化结构体指针	输出
agc_config	Agc 算法中当前的配置参数	输出

➤ 返回值

返回值	结果
0	成功
非 0	失败, 参考 错误码

➤ 依赖

- 头文件: AudioProcess.h
- 库文件: libAPC_LINUX.so/ libAPC_LINUX.a

※ 注意

无。

➤ 举例

请参考 [IaaAgc_Run](#) 举例部分。

3. APC 数据类型

3.1. APC 模块相关数据类型定义

数据类型	定义
AudioApcBufferConfig	Apc 算法的 buffer 配置参数结构体类型
AudioProcessInit	Apc 算法初始化参数结构体类型
APC_HANDLE	Apc 算法的句柄类型
AudioAnrConfig	Apc 算法中的 Anr 配置参数结构体类型
AudioEqConfig	Apc 算法中的 Eq 配置参数结构体类型
AudioHpfConfig	Apc 算法中的 Hpf 配置参数结构体类型
AudioAgcConfig	Apc 算法中的 Agc 配置参数结构体类型
AgcGainInfo	Apc 算法中 Agc 的增益参数结构体类型
IAA_APC_SAMPLE_RATE	Apc 算法定义的采样率类型
NR_CONVERGE_SPEED	Anr 算法的收敛速度类型
IAA_HPF_FREQ	Hpf 算法的截止频率类型
ANR_HANDLE	Anr 算法的句柄类型
EQ_HANDLE	Eq 算法的句柄类型
AGC_HANDLE	Agc 算法的句柄类型

3.2. AudioApcBufferConfig

➤ 说明

定义 Apc 算法的功能选项参数结构体。

➤ 定义

```
typedef struct{
    unsigned int anr_enable;
    unsigned int eq_enable;
    unsigned int dr_enable;
    unsigned int vad_enable;
    unsigned int agc_enable;
}AudioApcBufferConfig;
```

➤ 成员

成员名称	描述
anr_enable	Apc 算法中 Anr 使能选项，用于协助计算 Apc 算法运行所需要的内存大小
eq_enable	Apc 算法中 Eq 使能选项，用于协助计算 Apc 算法运行所需要的内存大小
dr_enable	算法等待开发中，目前没有效果，建议关闭
vad_enable	Vad 算法已建立独立模块，目前在 APC 中 Vad 已废弃，建议关闭
agc_enable	Apc 算法中 Agc 使能选项，用于协助计算 Apc 算法运行所需要的内存大小

※ 注意事项

无

➤ 相关数据类型及接口

[IaaApc_GetBufferSize](#)
[IaaApc_Init](#)
[IaaApc_Reset](#)

3.3. AudioProcessInit

➤ 说明

定义 Apc（Anr/Eq/Hpf/Agc）算法的初始化参数结构体。

➤ 定义

```
typedef struct {
    unsigned int point_number;
    unsigned int channel;
    IAA\_APC\_SAMPLE\_RATE sample_rate;
}AudioProcessInit;
```

➤ 成员

成员名称	描述
point_number	算法处理一次的数据量，即一个帧多少个采样点，值为 128 或 256 根据 chip 来决定需要给 Apc 多少个 short 值。 取值范围：128 或 256(根据 chip 决定)
channel	数据的声道数，若为双声道，则每帧给 Apc 的数据则为 point_number 的两倍。
sample_rate	数据的取样频率

※ 注意事项

无。

➤ 相关数据类型及接口

[IaaApc_Init](#)
[IaaApc_Reset](#)
[IaaApc_GetConfig](#)
[IaaAnr_Init](#)
[IaaAnr_Reset](#)
[IaaAnr_GetConfig](#)
[IaaEq_Init](#)
[IaaEq_Reset](#)
[IaaEq_GetConfig](#)
[IaaAgc_Init](#)
[IaaAgc_Reset](#)
[IaaAgc_GetConfig](#)

3.4. APC_HANDLE

➤ 说明

Apc 算法句柄类型。

➤ 定义

```
typedef void* APC_HANDLE;
```

➤ 成员

成员名称	描述
------	----

※ 注意事项

无。

➤ 相关数据类型及接口

[IaaApc_Init](#)
[IaaApc_Config](#)
[IaaApc_GetNrResult](#)
[IaaApc_GetNrEqResult](#)
[IaaApc_Run](#)
[IaaApc_Free](#)
[IaaApc_Reset](#)

3.5. AudioAnrConfig

➤ 说明

定义 Apc 算法中的 Anr 配置参数结构体类型

➤ 定义

```
typedef struct{
    unsigned int anr_enable;
    unsigned int user_mode;
    int anr_intensity_band[6];
    int anr_intensity[7];
    unsigned int anr_smooth_level;
    NR\_CONVERGE\_SPEED anr_converge_speed;
    int anr_filter_mode;
}AudioAnrConfig;
```

➤ 成员

成员名称	描述
anr_enable	是否使能 Anr 算法
user_mode	Anr 算法运行的模式
anr_intensity_band	降噪频率范围 范围[1,127]；步长 1
anr_intensity	降噪强度，值越大降噪强度越高，但同时也会带来细节的丢失/损伤，建议值:10。 范围[0,30]；步长 1
anr_smooth_level	频域平滑程度，建议值:10 范围[0,10]；步长 1
anr_converge_speed	噪声收敛速度，建议值:mid 范围[low;mid;high]
anr_filter_mode	频域降噪滤波器

	范围[0,4]; 步长 1
--	---------------

※ 注意事项

- 当 `anr_enable` 为 `FALSE` 时，其他的 `Anr` 参数也不会起作用。
- `user_mode` 指定 `Anr` 的运行模式。0，表示 `Anr` 算法完全不使用其他的 `Anr` 参数，而是使用 `Anr` 算法内部的设定；1 或 2 表示完全使用应用下的 `Anr` 参数
- `anr_intensity_band`，降噪频率范围，后一个元素必须大于等于前 1 个元素。

如：`u32NrIntensityBand[0] = 10`，则：`u32NrIntensityBand[1]`必须大于等于 10。

当前采样率对应的最高频率平均分成 128 份，频率范围则是对应多少份组成一个频带。

如：当前采样率为 16K，对应的最大频率为 8K，每一份为 $8000 / 128 \approx 62.5\text{Hz}$ 。如在 {4,6,36, 49,50,51} 的设定下，降噪频率范围为 {0~4 * 62.5Hz, 4~6 * 62.5Hz, 6~36 * 62.5Hz, 36~49 * 62.5Hz, 49~50 * 62.5Hz, 50~51 * 62.5Hz, 51-127 * 62.5Hz} = {0~250Hz, 250~375Hz, 375~2250Hz, 2250~3062.5Hz, 3062.5~3125Hz, 3125~3187.5Hz, 3187.5Hz~8000Hz}，`anr_intensity` 为降噪强度，可根据 `anr_intensity_band` 的频带划分，针对各个频带的噪声情况设置不同的参数。

- `anr_smooth_level`，`Anr` 算法处理频域的平滑程度，避免噪声估计时，在相邻频率上的压抑差距过大导致声音损伤。
- `anr_converge_speed`，`Anr` 算法的收敛速度，更新噪声的速度快慢，设定越快降噪收敛越快，但副作用是带来细节的丢失/损伤。
- `anr_filter_mode` 指定 `Anr` 算法。0 为原算法，1~4 为不同噪音估测之降噪算法。

➤ 相关数据类型及接口

[IaaApc_Config](#)
[IaaApc_GetConfig](#)
[IaaAnr_Config](#)
[IaaAnr_GetConfig](#)

3.6. AudioEqConfig

➤ 说明

定义 `Apc` 算法中的 `Eq` 配置参数结构体类型。

➤ 定义

```
typedef struct{
    unsigned int eq_enable;
    unsigned int user_mode;
    short eq_gain_db[129];
}
```



```
}AudioEqConfig;
```

➤ 成员

成员名称	描述
eq_enable	是否使能 Eq 算法
user_mode	Eq 算法的运行模式
eq_gain_db	Eq 算法增益调节取值，将当前采样率的频率范围分成 129 个频率范围来进行调节，单位为 1dB 范围[-50,20]；步长 1

※ 注意事项

- 当 eq_enable 为 FALSE 时，其他的 Eq 参数也不会起作用。
- user_mode 指定 Eq 的运行模式。0，表示 Eq 使用预设的参数，预设参数全为 0，不做任何增益或衰减；1，表示完全使用应用下的 Eq 参数。
- eq_gain_db 为增益调节的 table，将当前采样率的频率范围分成 129 个频率范围来进行调节。如：当前采样率为 16K，对应的最高频率为 8K， $8000 / 129 \approx 62\text{Hz}$ ，则单个调节的频率范围为 62Hz，将 0-8K 划分成 $\{0-1 * 62\text{Hz}, 1-2 * 62\text{Hz}, 2-3 * 62\text{Hz}, \dots, 128-129 * 62\text{Hz}\} = \{0-62\text{Hz}, 62-124\text{Hz}, 124-186\text{Hz}, \dots, 7938-8000\text{Hz}\}$ ，每段对应一个增益值。

➤ 相关数据类型及接口

[IaaApc_Config](#)
[IaaApc_GetConfig](#)
[IaaEq_Config](#)
[IaaEq_GetConfig](#)

3.7. AudioHpfConfig

➤ 说明

定义 Apc 算法中的 Hpf 配置参数结构体类型。

➤ 定义

```
typedef struct{
    unsigned int hpf_enable;
    unsigned int user_mode;
    IAA_HPF_FREQ cutoff_frequency;
}AudioHpfConfig;
```

➤ 成员

成员名称	描述
hpf_enable	是否使能 Hpf 算法
user_mode	Hpf 算法的运行模式
cutoff_frequency	Hpf 的截止频率 范围:[0,1,2,3,4]

※ 注意事项

- 当 hpf_enable 为 FALSE 时，其他的 Hpf 参数也不会起作用。
- user_mode 指定 Hpf 的运行模式。0，表示预设的参数，不做任何衰减；1，表示使用应用下的 Hpf 参数。
- cutoff_frequency 为 Hpf 的截止频率，低于此频率的信号会被过滤掉。

➤ 相关数据类型及接口

[IaaApc_Config](#)
[IaaApc_GetConfig](#)
[IaaEq_Config](#)
[IaaEq_GetConfig](#)

3.8. AudioAgcConfig

➤ 说明

定义 Apc 算法中的 Agc 配置参数结构体类型。

➤ 定义

```
typedef struct
{
    unsigned int agc_enable;
    unsigned int user_mode;
    //gain setting
    AgcGainInfo gain_info;
    unsigned int drop_gain_max;

    //attack time, release time
    unsigned int attack_time;
    unsigned int release_time;

    //target level
```

```

short compression_ratio_input[7];
short compression_ratio_output[7];
int drop_gain_threshold;

// noise gate
int noise_gate_db;
unsigned int noise_gate_attenuation_db;

unsigned int gain_step;
}AudioAgcConfig;

```

➤ 成员

成员名称	描述
agc_enable	是否使能 Agc 算法
user_mode	Agc 算法的运行模式
gain_info	Agc 算法的增益信息，定义 AGC 增益的最大、最小和初始值
drop_gain_max	瞬间增益下降的最大值，防止输出饱和，若输出加上当前 Gain 超出 drop_gain_threshold 所设定的 dB 值，Agc 会瞬间降低 Gain 避免当前信号的峰值超过 drop_gain_threshold。 范围[0,60]；步长 1 注意：此值仅代表能下降的范围，但具体能下降到何值还需参考 AGC 增益的最小值。
attack_time	增益下降的时间步长，以 4 毫秒为 1 单位，若设定 2 则为 8 毫秒判断一次是否降 Gain 范围[1,20]；步长 1
release_time	增益增加的时间步长，以 4 毫秒为 1 单位，若设定 2 则为 8 毫秒判断一次是否升 Gain 范围[1, 20]；步长 1
compression_ratio_input	配合 compression_ratio_output 使用，透过多个转折点实现多斜率的曲线，得到 input power level 跟 output power level 之间的关系。 范围[-80,0]dBFS；步长 1
compression_ratio_output	配合 compression_ratio_input 使用，透过多个转折点实现多斜率的曲线，得到 input power level 跟 output power level 之间的关系。 范围[-80,0]dBFS；步长 1
drop_gain_threshold	衰减阈值，当信号峰值幅度超过此值后，会瞬间衰减，衰减幅度受 drop_gain_max 跟 gain_info 所限制。 范围[-80,0]dB；步长 1
noise_gate_db	噪声阈值，当信号小于此值时，当作噪声处理， Case1: 若设定 noise_gate_db 从-80 到 0，当前 gain 值会根据 release/attack time 的时间将 Gain 值收敛成 0。

成员名称	描述
	Case2: 若设定 <code>noise_gate_db</code> 从 1 到 80, 当信号小于此值时, <code>Gain</code> 值将不会做更改, 会保留前一帧的 <code>Gain</code> 值。 范围[-80,80]; 步长 1
<code>noise_gate_attenuation_db</code>	当噪声阈值起效果时, 输入源的衰减百分比 范围[0,100]; 步长 1
<code>gain_step</code>	套用增益的速率, 以 0.5dB 为一个单位, 若设定为 1, 则每帧依照需求套用 $\pm 0.5\text{dB}$ 。 此值设定的越高, 拉升和降低音量的速率越快。 范围[1, 10]; 步长 1

※ 注意事项

- 当 `agc_enable` 为 `FALSE` 时, 其他的 `Agc` 参数也不会起作用。
- `user_mode` 指定 `Agc` 的运行模式。0, 表示 `Agc` 算法完全不使用其他的 `Agc` 参数, 而是使用 `Agc` 算法内部的设定; 1, 表示完全使用应用下的 `Agc` 参数。
- `Compression ratio Curve` 定义输入输出能量之间的关系, 透过设定七个点可以得到六段不同的斜率转折, 举例来说, 参数设定如下:

`compression_ratio_input[7] = {-80, -60, -50, -40, -30, -12, 0}`

`compression_ratio_output[7] = {-80, -45, -36, -27, -18, -9, -6}`

得到的曲线如下图所示

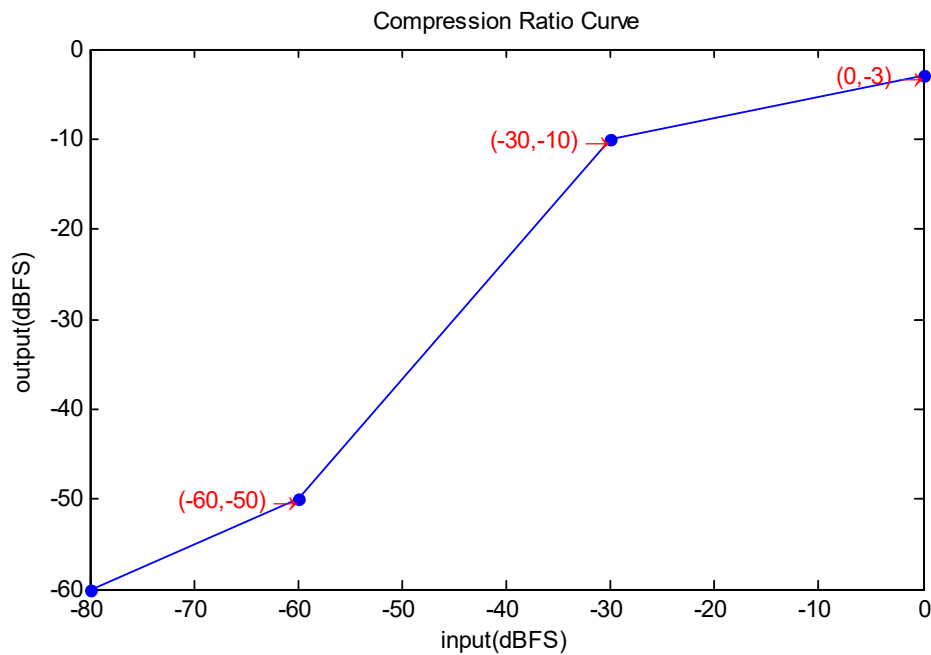
input (dBFS)

若不需这么多段不同斜率, 可以将最后的 `compression ratio` 参数设成 0, 如下参数

`compression_ratio_input[7] = {-70, -60, -30, 0, 0, 0, 0};`

`compression_ratio_output[7] = {-60, -50, -10, -3, 0, 0, 0};`

曲线如下图



- Noise gate 设定有两种模式，可以将 Gain 收敛到 0 或维持 Gain 值不变，收敛到 0 的状况可避免小于 noise gate 的信号被放大，维持噪声稳定度，维持 Gain 值不变可避免语音的起头与尾音消失的”呼吸现象”。

➤ 相关数据类型及接口

[IaaApc_Config](#)
[IaaApc_GetConfig](#)
[IaaAgc_Config](#)
[IaaAgc_GetConfig](#)

3.9. AgcGainInfo

➤ 说明

定义 Apc 算法中 Agc 的增益参数结构体类型。

➤ 定义

```
typedef struct
{
    int gain_max; //gain maximum
    int gain_min; //gain minimum
    int gain_init; //default gain (initial gain)
}AgcGainInfo;
```

➤ 成员

成员名称	描述
gain_max	增益最大值 范围[0,60]；步长 1
gain_min	增益最小值 范围[-20,30]；步长 1
gain_init	增益初始值 范围[-20,60]；步长 1

※ 注意事项

无。

➤ 相关数据类型及接口

[AudioAgcConfig](#)

3.10. IAA_APC_SAMPLE_RATE

➤ 说明

定义 Apc 算法中的采样率类型。

➤ 定义

```
typedef enum {  
    IAA_APC_SAMPLE_RATE_8000 = 8000 ,  
    IAA_APC_SAMPLE_RATE_16000 = 16000 ,  
    IAA_APC_SAMPLE_RATE_48000 = 48000 ,  
}IAA_APC_SAMPLE_RATE;
```

➤ 成员

成员名称	描述
IAA_APC_SAMPLE_RATE_8000	8K 采样率
IAA_APC_SAMPLE_RATE_16000	16K 采样率
IAA_APC_SAMPLE_RATE_48000	48K 采样率

※ 注意事项

无。

- 相关数据类型及接口

[AudioProcessInit](#)

3.11. NR_CONVERGE_SPEED

- 说明

定义 Apc 算法中 Anr 收敛速度类型。

- 定义

```
typedef enum {
    NR_SPEED_LOW,
    NR_SPEED_MID,
    NR_SPEED_HIGH
} NR_CONVERGE_SPEED;
```

- 成员

成员名称	描述
NR_SPEED_LOW	低速
NR_SPEED_MID	中速
NR_SPEED_HIGH	高速

- ※ 注意事项

- NR 收敛速度低/中/高分别代表 3/2/1 帧更新一次噪声压抑

- 相关数据类型及接口

[AudioAnrConfig](#)

3.12. IAA_HP_FREQ

- 说明

定义 Apc 算法中 Hpf 截止频率类型。

- 定义

```
typedef enum {
    AUDIO_HP_FREQ_80 , /* 80Hz*/
```

```

    AUDIO_HPF_FREQ_120, /*120Hz*/
    AUDIO_HPF_FREQ_150, /*150Hz*/
    AUDIO_HPF_FREQ_500, /*500Hz*/
    AUDIO_HPF_FREQ_BUTT,
}IAA_HPF_FREQ;

```

➤ 成员

成员名称	描述
AUDIO_HPF_FREQ_80	80Hz
AUDIO_HPF_FREQ_120	120Hz
AUDIO_HPF_FREQ_150	150Hz
AUDIO_HPF_FREQ_500	500Hz
AUDIO_HPF_FREQ_BUTT	

※ 注意事项

无。

➤ 相关数据类型及接口

[AudioHpfConfig](#)

3.13. ANR_HANDLE

➤ 说明

定义 Anr 算法的句柄类型。

➤ 定义

```
typedef void* ANR_HANDLE;
```

➤ 成员

成员名称	描述
------	----

※ 注意事项

无。

➤ 相关数据类型及接口

[IaaAnr_Init](#)
[IaaAnr_Config](#)
[IaaAnr_Run](#)
[IaaAnr_Free](#)
[IaaAnr_Reset](#)
[IaaAnr_GetConfig](#)

3.14. EQ_HANDLE

➤ 说明

定义 Eq 算法的句柄类型。

➤ 定义

```
typedef void* EQ_HANDLE;
```

➤ 成员

成员名称	描述
------	----

※ 注意事项

无。

➤ 相关数据类型及接口

[IaaEq_Init](#)
[IaaEq_Config](#)
[IaaEq_Run](#)
[IaaEq_Free](#)
[IaaEq_Reset](#)
[IaaEq_GetConfig](#)

3.15. AGC_HANDLE

➤ 说明

定义 Agc 算法的句柄类型。

➤ 定义

```
typedef void* AGC_HANDLE;
```

➤ 成员

成员名称	描述
------	----

※ 注意事项

无。

➤ 相关数据类型及接口

[IaaAgc_Init](#)
[IaaAgc_Config](#)
[IaaAgc_Run](#)
[IaaAgc_Free](#)
[IaaAgc_Reset](#)
[IaaAgc_GetConfig](#)

4. 错误码

APC API 错误码如下表所示：

表 4-1 APC API 错误码

错误码	宏定义	描述
0x00000000	ALGO_APC_RET_SUCCESS	APC 运行成功
0x10000501	ALGO_APC_RET_INIT_ERROR	APC 尚未初始化
0x10000502	ALGO_APC_RET_INVALID_HANDLE	HANDLE 无效
0x10000503	ALGO_APC_RET_INVALID_SAMPLE_RATE	取样频率不支持
0x10000504	ALGO_APC_RET_INVALID_POINT_NUMBER	每帧点数不支援
0x10000505	ALGO_APC_RET_INVALID_CHANNEL	通道数不支持
0x10000506	ALGO_APC_ANR_RET_INVALID_ENABLE	ANR 开关参数设置无效
0x10000507	ALGO_APC_ANR_RET_INVALID_MODE	ANR 模式参数设置无效
0x10000508	ALGO_APC_ANR_RET_INVALID_INTENSITY	ANR 强度参数设置无效
0x10000509	ALGO_APC_ANR_RET_INVALID_SMOOTH_LEVEL	ANR 平滑化参数设置无效
0x10000510	ALGO_APC_ANR_RET_INVALID_COVERGE_SPEED	ANR 收敛速度参数设置无效
0x10000511	ALGO_APC_EQ_RET_INVALID_ENABLE	EQ 开关参数设置无效
0x10000512	ALGO_APC_EQ_RET_INVALID_MODE	EQ 模式参数设置无效
0x10000513	ALGO_APC_EQ_RET_INVALID_TABLE	EQ 参数设置无效
0x10000514	ALGO_APC_HPF_RET_INVALID_ENABLE	HPF 开关参数设置无效

错误码	宏定义	描述
0x10000515	ALGO_APC_HPF_RET_INVALID_MODE	HPF 模式参数设置无效
0x10000516	ALGO_APC_HPF_RET_INVALID_TABLE	HPF 参数设置无效
0x10000517	ALGO_APC_AGC_RET_INVALID_ENABLE	AGC 开关参数设置无效
0x10000518	ALGO_APC_AGC_RET_INVALID_MODE	AGC 模式参数设置无效
0x10000519	ALGO_APC_AGC_RET_INVALID_COMPRESSION_RATIO	AGC 压缩曲线参数设置无效
0x10000520	ALGO_APC_AGC_RET_INVALID_FREQUENCY_BAND	AGC 频段设置无效
0x10000521	ALGO_APC_AGC_RET_INVALID_DROP_GAIN_MAX	AGC 防饱和参数设置无效
0x10000522	ALGO_APC_AGC_RET_INVALID_GAIN_STEP	AGC 套用增益速率设置无效
0x10000523	ALGO_APC_AGC_RET_INVALID_RELEASE_TIME	AGC 复原时间参数设置无效
0x10000524	ALGO_APC_AGC_RET_INVALID_ATTACK_TIME	AGC 削减时间参数设置无效
0x10000525	ALGO_APC_AGC_RET_INVALID_NOISE_GATE	AGC 噪声阈值参数设置无效
0x10000526	ALGO_APC_AGC_RET_INVALID_NOISE_ATTENU	AGC 噪声衰减设置无效
0x10000527	ALGO_APC_AGC_RET_INVALID_DROP_GAIN_LEVEL	AGC 防饱和阈值参数设置无效
0x10000528	ALGO_APC_AGC_RET_INVALID_GAIN_INFO	AGC 增益限制参数设置无效
0x10000529	ALGO_APC_RET_API_CONFLICT	其他 API 正在运行
0x10000530	ALGO_APC_RET_INVALID_CALLING	呼叫 API 顺序错误
0x10000531	ALGO_APC_RET_FAILED	参数设置错误